

NUFFT_{CF}: FAST AUTO- AND CROSS-CORRELATION FUNCTION ESTIMATION FOR IRREGULARLY-SAMPLED TIME SERIES VIA THE NON-UNIFORM FFT

JEAN-ERIC CAMPAGNE 

Université Paris-Saclay, CNRS/IN2P3, IJCLab, 91405 Orsay, France

Version August 3, 2026

Abstract

Estimating the auto-correlation function (ACF) of a single, and the cross-correlation function (CCF) between two, irregularly-sampled scalar time series is a standard task across different physics fields as astronomy, hydrogeology, and environmental fields. We aim to provide an ACF/CCF estimator that is numerically consistent with established, well-validated kernel-weighted definitions, while scaling as $O(n \log n)$ rather than $O(n^2)$.

nufftcf evaluates the Wiener–Khinchin theorem for irregularly sampled data using the Non-Uniform Fast Fourier Transform (NUFFT), via the Flatiron Institute’s **FINUFFT** library, and using the Gaussian- and rectangular-kernel estimators. An $O(n)$ two-pointer scan replaces the naive $O(n^2)$ computation of the effective pair count that normalizes each lag bin. Direct, real-space implementations of the same estimators, and dedicated classical-FFT estimators for regularly-sampled data, are provided alongside the NUFFT route as accuracy references, all sharing a single calling convention.

We validate **nufftcf** against **pastas** (regular-grid ACF, to numerical precision) known in groundwater time series analysis and against **pyZDCF** (CCF, on cases with known theoretical cross-correlation) known in astronomical time series analysis. We measure a speed-up of approximately [XX]x over **pastas** for ACF estimation at $n = [XX]$, with the NUFFT estimator’s residual spectral-leakage bias quantified at [XX]% for strongly periodic, gappy input. **[TODO: summarize validation and benchmark results here.]**

Subject headings: Time series analysis, Correlation function, Non-uniform Fast Fourier Transform

1. INTRODUCTION

Many measurements of physical, environmental, and astrophysical systems take the form of a scalar time series that is not regularly sampled in time. Ground-based photometric monitoring of active galactic nuclei (AGN) is interrupted by weather, daylight, and telescope scheduling; groundwater levels are logged during irregular field visits; paleoclimate proxies are deposited at a rate that itself varies with the climate signal being measured. A recurring analysis task for such data is to estimate the autocorrelation function (ACF) of a single series, or the cross-correlation function (CCF) between two independently and irregularly sampled series, at a set of time lags τ . Applications range from measuring a characteristic variability or persistence timescale, to detecting a time delay between two related signals. For instance in AGN reverberation mapping, where the CCF between a driving continuum light curve and a reprocessed emission-line light curve encodes the size of the emitting region (Welsh 2024; Jankov et al. 2022a; Sánchez-Sáez et al. 2021; Shapovalova et al. 2019; Kovačević et al. 2014), one uses the discrete correlation function (Edelson & Krolik 1988) and also for instance the python version (**pyZDCF**) of the code by Alexander (1997).¹

Because the classical, textbook estimator of the ACF/CCF assumes a common, regular sampling grid (e.g. Stoffer 2026), irregular sampling has historically been handled with different technics (see Kreutzer et al. 2023; Rehfeld et al. 2011): resampling the data onto a

regular grid by interpolation before applying the standard Fourier-based estimator; binning observation pairs by their time-lag separation (“slotting”); weighting observation pairs by a smooth kernel of the time-lag separation rather than binning them; or bypassing the lag domain altogether and working with a Fourier-domain estimator adapted to irregular sampling, such as the Lomb–Scargle periodogram (VanderPlas 2018; Scargle 1982; Lomb 1976).

Each of these approaches has been implemented, refined, and benchmarked by different research communities—astronomy, geoscientific context among them—largely independently of one another (Section 2). What these approaches share is a computational cost that scales at best linearly per output lag and, in the pairwise binning/weighting case, quadratically overall in the number of observations n . This is unproblematic for the short, sparse times series where these methods were originally designed for (typically $n \sim 10^2$ – 10^3), but certainly would become a practical bottleneck for the longer, denser irregular time series increasingly produced by continuous environmental monitoring, high-cadence time-domain surveys, or long instrumental records ($n \sim 10^4$ – 10^6), and for workflows that require many repeated evaluations (parameter sweeps, bootstrap resampling, batch processing of large numbers of light curves or well records).

In this paper we present **nufftcf**, an open-source Python package that computes ACF/CCF estimates by evaluating the Wiener–Khinchin theorem with the Non-Uniform Fast Fourier Transform (NUFFT) and classical-FFT for irregularly- and regularly-sampled time series re-

jean-eric.campagne@ijclab.in2p3.fr

¹ Description of other use-cases in **Pastas** and elsewhere.

spectively. This leads to the cost reduction of the kernel-weighted estimators in widest use (e.g. the Gaussian-kernel estimator used by the `pastas` hydrogeological time-series package) from $O(n^2)$ to $O(n \log n)$. `nufftcf` additionally provides direct, “real-space” implementations of the same estimators as an accuracy reference. All estimator families share a single calling convention, so that a user can trade accuracy against speed without altering downstream analysis code.

The development of `nufftcf` yields the following contributions:

1. We demonstrate that the Gaussian- and rectangular-kernel ACF/CCF estimators—previously established on statistical grounds (Rehfeld et al. 2011)—can be reformulated as a pair of Non-Uniform Fast Fourier Transform (NUFFT) evaluations. This reformulation ensures numerical consistency with existing definitions while achieving an $O(n \log n)$ computational complexity.
2. We introduce an $O(n)$ analytical algorithm, based on a two-pointer scan, to compute the effective pair count for normalizing each lag bin. This replaces the naive $O(n^2)$ all-pairs approach, significantly improving efficiency.
3. We validate `nufftcf` through comparisons with `pastas` (Collenteur et al. 2019) and `pyZDCF` (Jankov et al. 2022b) on synthetic ACF and CCF problems, and quantify the resulting speed-up.

The paper is organized as follows. Section 2 reviews prior approaches to ACF/CCF estimation for irregularly-sampled data, situates `nufftcf` relative to them, and details the methodology, from the regularly-sampled case through the four established families of irregular-sampling estimators to the NUFFT reformulation used by `nufftcf`. Section 3 compares `nufftcf` to `pastas` and `pyZDCF` on ACF and CCF use cases. Section 4 presents timing benchmarks. Section 5 discusses limitations and guidance on estimator choice, and Section 6 concludes.

2. RELATED WORKS AND METHODOLOGY

2.1. Correlation estimation on a regular grid: the Wiener–Khinchin theorem

Let $x_t, t = 0, \dots, n-1$, be a zero-mean, weakly stationary process sampled at N regularly spaced times with sampling interval Δt . The (biased) sample autocovariance at lag $k\Delta t$ is

$$\hat{R}_{xx}(k) = \frac{1}{n} \sum_{t=0}^{n-k-1} x_t x_{t+k}, \quad k = 0, \dots, n-1, \quad (1)$$

and the sample ACF is $\hat{\rho}_{xx}(k) = \hat{R}_{xx}(k)/\hat{R}_{xx}(0)$. The sample cross-covariance and cross-correlation between two co-sampled series x_t and y_t are defined analogously.

For weakly stationary processes, the Wiener–Khinchin theorem states that the autocovariance (or cross-covariance) function and the power (or cross-)spectral density form a Fourier-transform pair. For finite sampled data, this relation is realized numerically through the discrete Fourier transform (DFT): the cross-spectrum is estimated as

$$\hat{S}_{xy}(f) = \hat{X}(f)\hat{Y}^*(f), \quad \hat{R}_{xy}(k) = \mathcal{F}^{-1} \left[\hat{S}_{xy}(f) \right] (k), \quad (2)$$

where $\hat{X}(f) = \mathcal{F}[x](f)$ (and similarly for $\hat{Y}(f)$) denotes the DFT of the sampled series. On a regular grid, this identity is evaluated efficiently using a pair of FFTs—a forward FFT to compute $\hat{X}(f)$ and $\hat{Y}(f)$, a pointwise complex-conjugate product to form $\hat{S}_{xy}(f)$, and an inverse FFT to recover $\hat{R}_{xy}(k)$ —yielding an $O(n \log n)$ algorithm instead of the $O(n^2)$ direct lag-domain summation of Equation 1.

This FFT-based approach provides the regular-grid baseline for correlation estimation. However, for irregularly-sampled data, where the sample times $t_1 < t_2 < \dots < t_n$ are not equally spaced, neither the sum in Equation 1 nor the FFT pair can be applied directly. The literature describes four broad strategies to address this issue, which we outline below.

2.2. Slotting and the discrete correlation function

The earliest widely used approach specific to irregular sampling is the *slotting* technique, introduced in astronomy by Edelson & Krolik (1988) as the discrete correlation function (DCF). Slotting bins the products of pairs of (standardized) observations by their time-lag separation into fixed-width bins and averages within each bin. While simple and model-free, the resulting correlation-function estimate is not guaranteed to be positive semi-definite and may require post-processing (Rehfeld et al. 2011). This limitation has motivated the development of kernel-weighted techniques (Section 2.3).

A refinement of the DCF, proposed by Alexander (1997) (ZDCF-transform), addresses the sparse, unevenly sampled light curves typical of AGN monitoring by using *equal-population binning* rather than fixed-width bins. In this approach, bin boundaries are chosen adaptively so that each bin contains approximately an equal number of pairs. Additionally, a Fisher z -transform (Fisher 1915) is applied to stabilize the variance of the correlation estimate in each bin before computing confidence intervals. `pyZDCF` (Jankov et al. 2022b) is the reference Python implementation of ZDCF and remains the standard tool for CCF estimation in AGN reverberation-mapping studies. Both the DCF and ZDCF require a pairwise scan of the data, resulting in an $O(n^2)$ computational cost.

2.3. Kernel-weighted estimators

Kernel-weighted estimators replace the hard bin assignment of slotting with a smooth weighting function (kernel) of the time-lag separation. This allows pairs to contribute to a lag estimate in proportion to the proximity of their separation to that lag. In this context, Stoica et al. (2009) proposed a sinc kernel, while Bjørnstad & Falck (2001) introduced a Gaussian kernel. Notably, the slotting technique of Edelson & Krolik (1988) can be interpreted as a rectangular (or box) kernel.

The kernel-weighted cross-correlation estimator is de-

defined as:

$$\hat{\rho}_{xy}(k\Delta\tau) = \frac{\sum_{i=1}^{n_x} \sum_{j=1}^{n_y} x_i y_j b_k(d)}{\sum_{i=1}^{n_x} \sum_{j=1}^{n_y} b_k(d)}, \quad (3)$$

where $d = t_j^y - t_i^x - k\Delta\tau$ represents the deviation from the target lag. Typical choices for $b_k(d)$ include the rectangle/box kernel (Edelson & Krolik 1988):

$$b_k(d) = \begin{cases} 1 & \text{for } |d| \leq h/2, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

and the Gaussian kernel (Björnstad & Falck 2001), which is the primary choice in `pastas` and `nufftcf`:

$$b_k(d) = \frac{1}{\sqrt{2\pi}h} \exp\left(-\frac{d^2}{2h^2}\right). \quad (5)$$

Here, h is a bandwidth parameter typically set to match the mean sampling interval Δt^{xy} (e.g., $h = \Delta t^{xy}/2$ for the rectangle kernel, $h = \Delta t^{xy}/4$ for the Gaussian kernel; see (Rehfeld et al. 2011) for details).

Given the irregular sampling common in geoscientific time series, Rehfeld et al. (2011) conducted a systematic benchmark of kernel-based estimators (Gaussian, sinc, rectangular) against linear interpolation and the Lomb–Scargle approach. Using synthetic sinusoidal and autoregressive processes with controlled sampling irregularity, they found that the Gaussian-kernel estimator consistently exhibited the lowest, or close to the lowest, root-mean-square error and bias across nearly all test cases. This led the authors to recommend it over linear interpolation for irregularly sampled data.

This methodology underpins the ACF estimator implemented in `pastas` (Collenteur et al. 2019), a widely used Python package for groundwater time-series analysis. `pastas` offers Gaussian, rectangular, and regular-grid binning options for its ACF. As with slotting, kernel-weighted estimators evaluate a direct pairwise sum, with an overall cost of $O(n^2)$ for a full lag range (or $O(n)$ per individual lag).

2.4. Lomb–Scargle-based (Fourier-domain) estimation

A conceptually different approach, introduced by Lomb (1976); Scargle (1982) (see also Scargle (1989); VanderPlas (2018)), estimates the cross-correlation function by first computing the cross-spectrum via the Lomb–Scargle Fourier transform (LSFT). The LSFT is an irregular-sampling generalization of the DFT built from a least-squares sinusoid fit. The (cross-)periodogram is then formed, and an inverse Fourier transform is applied to the result, effectively applying the Wiener–Khinchin theorem in the Fourier domain rather than directly binning or weighting observation pairs in the lag domain.

Rehfeld et al. (2011) found this approach competitive with kernel methods for univariate (ACF) problems but markedly worse for bivariate (CCF) problems. While the Lomb–Scargle method reuses the transform-multiply-invert idea of Section 2.1, it replaces the FFT with a nonuniform transform tailored to irregular sampling, with a computational cost of $O(n n_f)$ for n_f frequencies.

2.5. The Non-Uniform Fast Fourier Transform

The NUFFT computes Fourier sums between nonuniformly and uniformly spaced points without forming the $O(n \cdot m)$ direct sum over all n sample points and m output frequencies (or vice versa). Two of its three standard types are relevant here (see sign convention in Section A.3):

- **Type 1** (“nonuniform to uniform”): given values x_j at nonuniform points t_j , $j = 1, \dots, n$, compute

$$\hat{X}(f_k) = \sum_j x_j e^{if_k t_j} := X_k \quad (6)$$

at m uniform output frequencies f_k .

- **Type 2** (“uniform to nonuniform”): the adjoint operation, evaluating a uniformly-gridded Fourier series at arbitrary nonuniform output points:

$$x_j = \sum_k X_k e^{-if_k t_j} \quad (7)$$

Modern NUFFT algorithms, including FINUFFT (Barnett et al. 2019) which `nufftcf` uses, compute both types in $O((n + m) \log(1/\varepsilon))$ time for a requested accuracy ε , by (i) *spreading* each nonuniform sample onto a fine uniform grid using a compactly-supported interpolation kernel (FINUFFT uses an “exponential of semicircle” kernel chosen to minimize aliasing error for a given support width), (ii) taking a standard FFT of the fine grid, and (iii) *deconvolving*—dividing by the Fourier transform of the spreading kernel—to correct for the smoothing the kernel introduced. Type 2 reverses this sequence. This spreading kernel is an internal numerical-accuracy device of the NUFFT algorithm itself, distinct from the statistical weighting kernels $b_k(\cdot)$ of Equation 5. We return below to the distinction between the two kernels (spreading and statistical weighting).

2.6. From kernel-weighted estimators to NUFFT: the `nufftcf` approach

`nufftcf`’s central observation is that the kernel-weighted estimator of Equation 3 can be evaluated via Wiener–Khinchin using a NUFFT, rather than via a direct pairwise sum, whenever the weighting kernel $b_k(\cdot)$ is itself expressible as (or well approximated by) a smooth, compactly supported function of the lag—which the Gaussian and rectangular kernels used by `pastas` are. Concretely, `nufftcf`:

1. forms the (irregularly sampled) power or cross-spectrum of the input series with a **type-1 NUFFT** onto a uniform frequency grid whose resolution is set by the requested lag range and kernel bandwidth;
2. evaluates the correlation estimate at the requested lags by inverting this spectrum with a **type-2 NUFFT**, applying the Wiener–Khinchin theorem in the same spirit as the regular-grid FFT pair of Section 2.1, but with both legs replaced by their nonuniform-capable counterparts;

Algorithms 1–3 in Appendix A give the complete pseudocode of the CCF and ACF estimators, respectively, and Appendix A.3 details the implementation choices dictated by FINUFFT’s conventions (domain mapping, sign convention, oversampling factor N_1 , and the interior-lag normalization used to avoid an edge-smoothing bias). The total cost of evaluating the correlation function at K lags from n (irregular) samples is $O(n \log n + K \log K)$, instead of the $O(n^2)$ (or $O(nK)$) cost of the direct sum in Equation 3. Because this route implicitly represents the signal on a finite, periodic Fourier basis, it is subject to a small spectral-leakage bias for strongly periodic input with irregular or gappy sampling (quantified in Section 4 and controllable via the number of Fourier modes); `nufftcf` therefore also ships a direct, “real-space” evaluation of the same kernel-weighted estimator, at the original $O(n^2)$ cost, both as an accuracy reference and as the recommended choice when n is modest and this bias must be avoided entirely (Section 5).

Effective pair count. Every estimate in Equation 3 is normalized by $b_k(\cdot)$ summed over all pairs—the *effective number of pairs* contributing to lag k , which both normalizes the correlation estimate and flags under-sampled lags. Computed naively this denominator is itself an $O(n^2)$ all-pairs sum. Because the sample times are sorted, `nufftcf` instead computes it in $O(n)$ with a two-pointer scan implemented as a `numba`-compiled (Lam et al. 2015) kernel per supported weighting kernel (Algorithms 4–5 in Appendix A.2), and, for the regular-grid classic-FFT estimators, recovers the same quantity for free by filtering the deterministic raw-pair-count ramp with the same discrete filter applied to the correlation numerator. This avoids reintroducing an $O(n^2)$ step alongside an otherwise $O(n \log n)$ estimator.

Unified API. All three estimator families implemented in `nufftcf`—NUFFT-based, real-space, and classic-FFT (regular grid only)—share the calling convention `fn(lags, tx, x, [ty, y], ...)` → `(c, b)`², so that a user can move between them, e.g. to check a NUFFT-based result against the real-space reference on a subset of the data, without altering the rest of an analysis pipeline.

3. COMPARISON WITH PASTAS AND PYZDCF

We now validate `nufftcf` against two widely used, independently implemented estimators — `pastas` (kernel-weighted, real-space, Section 2.3) and `pyZDCF` (equal-population slotting, Section 2.2) — on synthetic series for which the “true” correlation function is known analytically. We treat ACF and CCF separately, since `pastas` does not expose a cross-correlation estimator equivalent to its ACF³, so the CCF benchmark below compares only `nufftcf` against `pyZDCF`.

3.1. Test series and experimental setup

² `(c, b)` standing for “correlation estimate” and “effective pair count”.

³ An internal cross-correlation code exists from which the ACF is computed, but it is not part of its public API. In the `pastas/pastas-plugins` repository, the cross-correlation code uses the `scipy.signal.correlate` function (Virtanen et al. 2020) with the FFT method, which is only valid for equidistant time steps.

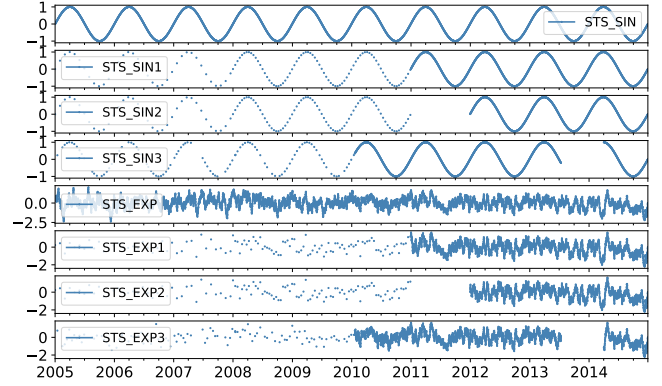


FIG. 1.— The synthetic series used in the ACF benchmark: the fully-sampled reference processes (STS_SIN, STS_EXP) and their three irregularly-sampled derivatives each (STS_SIN1–3, STS_EXP1–3), reproduced from the `pastas` autocorrelation-benchmark tutorial.

ACF benchmark series⁴. We reuse the synthetic irregularly-sampled series from the `pastas` autocorrelation-benchmark tutorial⁵, shown in Figure 1. Two underlying stationary processes are generated over a ten-year span at a one-day base resolution:

- **STS_SIN:** a pure sine wave of period $T = 365.25$ d, with analytical autocorrelation $\rho(\tau) = \cos(2\pi\tau/T)$;
- **STS_EXP:** an AR(1)-like process obtained by convolving white noise with a causal exponential kernel $e^{-t/\alpha}$, $\alpha = 10$ d; because this kernel coincides with the process autocovariance, its analytical ACF is $\rho(\tau) = e^{-\tau/\alpha}$.

Each process is then resampled onto three irregular time grids of different sparsity, yielding six benchmark series (STS_SIN-3, STS_EXP1-3): (1) a mix of 30-day and 14-day regular sub-sampling with dense daily coverage over the last four years; (2) the same scheme with an additional three-year data gap; and (3) re-indexing onto a set of real, highly irregular groundwater-monitoring dates spanning 1960–2015, used as is from the `pastas` tutorial. Series 3 is by far the sparsest and most irregular of the three ($n \sim 500600$ points over 55 years with long gaps), while series 1–2 alternate between decades of sparse sampling and a few years of dense daily coverage.

CCF benchmark series⁶. Two complementary CCF setups are used, both without access to `pastas`:

- **Shifted-timestamp STS_SIN/EXP series:** The same STS_SIN and STS_EXP processes used in the ACF benchmark are drawn over $n_{\text{days}} = 3650$ d, and two independent, irregular observation networks $x(t_x)$ and $y(t_y)$ sample the identical underlying process values, with y -timestamps shifted by a known delay τ_0 relative to x -timestamps ($\sim 43\%$ of days retained in each series). Because x and y are the same signal read at shifted times, the CCF should peak exactly at $\tau = \tau_0$ with the ACF own peak value (1 for these unit-variance processes). We use

⁴ see `pastas_vs_nufftcf.ipynb` and `zdcf_vs_nufftcf.ipynb` notebooks.

⁵ <https://pastas.readthedocs.io/stable/benchmarks/autocorrelation.html>

⁶ see `nufftcf_ccf_demo.ipynb` notebook.

$\tau_0 = 60$ d as the fiducial case (Figure 5) and additionally sweep $\tau_0 \in \{20, 60, 120\}$ d for the STS_EXP series (Figure 6) to check that the recovered peak location tracks the injected delay.

- Coupled Ornstein–Uhlenbeck (OU) processes with variable ρ . Two independent latent OU-like processes $e_1(t)$, $e_2(t)$ are generated with the same exponential-kernel construction as STS_EXP ($\alpha = 10$ d), and combined as

$$\begin{aligned} X_1(t) &= e_1(t), \\ X_2(t) &= \rho e_1(t - \tau_0) + \sqrt{1 - \rho^2} e_2(t), \end{aligned} \quad (8)$$

with $\tau_0 = 60$ d and $\rho \in \{0.1, 0.4, 0.7, 0.95\}$. Unlike the shifted-timestamp case, here the lag is *physical*: X_2 is a genuinely delayed, partially decorrelated copy of X_1 , contaminated by an independent noise process. The two series are sampled on independent irregular grids retaining $\sim 40\%$ and $\sim 60\%$ of days, respectively, with no artificial timestamp shift. The analytical cross-correlation function for this construction is

$$\rho_{X_1 X_2}(\tau) = \rho e^{-|\tau - \tau_0|/\alpha}, \quad (9)$$

i.e. an exponential peak of height ρ centered at $\tau = \tau_0$ (Figure 7).

Software configuration. For `nufftcf` we evaluate both the rectangular and Gaussian kernels (Equations 4, 5) via the NUFFT-based estimator (and, for the fiducial CCF case, the direct real-space evaluation as an accuracy cross-check), using `bin_width=0.5` d as the kernel-bandwidth control parameter for all ACF and CCF runs, and evaluating every integer lag from 1 to 365 d (ACF) or 1 to 180 d (CCF).

For `pastas` we call `pastas.stats.acf` with `bin_method` set to "rectangle" and "gaussian" and `max_gap=30` d, over the same lag range.

For `pyZDCF` we use `uniform_sampling = False`, `omit_zero_lags = True`, and its equal-population bin-size parameter `minpts`, the closest analogue to `nufftcf` and `pastas` bandwidth: we report `minpts = 11` (its default in practice⁷) for all CCF runs and both `minpts = 11` and `minpts = 25` for the ACF runs, with asymmetric Monte-Carlo error bars from 50 (ACF) or 100 (CCF) resamplings. Because `pyZDCF` adapts its bin boundaries to the pair count rather than to a fixed lag spacing, the number of lags it returns is set by the data and by `minpts`, i.e., not chosen by the user. It therefore always produces *far fewer* lag points than the fixed grid used by `nufftcf`, typically resulting in $\sim 100 - 500$ bins for the ACF series here, compared to the full requested grid of 365 or 180 points for `nufftcf`. This is a direct consequence of `pyZDCF` equal-population-per-bin design (Section 2.2).

3.2. Autocorrelation: `nufftcf` vs `pastas` and `pyZDCF`

Figures 2–3 compare `nufftcf` and `pastas` on the STS_SIN and STS_EXP series, respectively, showing the estimated ACF together with the residual to the analytical truth for both the Gaussian and rectangular kernels.

⁷ In practice, the default `minpts = 0` in `pyZDCF` is internally replaced by `ENOUGH = 11` (see `pyzdcf/pyzdcf.py`).

Across all six series the two implementations are visually indistinguishable and their Root Mean Square Error (RMSE) against the analytical ACF agree to within the reported precision (e.g. RMSE = 0.03/0.05/0.04 for STS_SIN1/2/3 and RMSE = 0.08/0.11/0.07 for STS_EXP1/2/3, identical for `pastas` and `nufftcf` in every case). This confirms that `nufftcf`, NUFFT-based Wiener–Khinchin methodology, reproduces the direct pairwise kernel-weighted estimator of Equation 3 to the accuracy expected from its spectral-leakage bias (Section 2.6), which here is negligible compared to the sampling and estimation noise common to both methods. The residuals for both methods grow with n -dependent gaps and departures from stationarity (e.g. around the transition between sparse and dense sampling in STS_EXP1/2 near lag $\sim 40 - 100$ d), which is a property of the underlying data and kernel bandwidth rather than of either estimator.

Figure 4 repeats the comparison against `pyzdcf`. The `nufftcf` Gaussian and rectangular estimates, which closely follow the analytical ACF, agree with the `pyzdcf` points (within their Monte-Carlo error bars) wherever the two methods are directly comparable. The main qualitative differences are in resolution and computational cost: `nufftcf` returns a value at every requested lag for these series (e.g., 365 points) in at most a few hundred milliseconds, whereas `pyzdcf` returns only $\sim 300 - 500$ equal-population bins (even fewer for the coarser `minpts = 25` setting) and requires several seconds per series. These timing differences are driven by `pyzdcf` $O(n^2)$ pairwise scan and Monte Carlo error resampling, compared to `nufftcf` $O(n \log n)$ NUFFT evaluation.

3.3. Cross-correlation: `nufftcf` vs `pyZDCF`

Figure 5 show the fiducial $\tau_0 = 60$ d shifted-timestamp case for the STS_SIN and STS_EXP series, respectively, comparing `nufftcf` Gaussian estimate against its own real-space reference (left panels) and against its rectangular-kernel estimate (right panels), with `pyZDCF` estimate overlaid in both. The three `nufftcf` variants agree essentially exactly with one another, and all three track `pyZDCF` sparser set of binned points within its error bars, correctly locating the CCF peak at the injected delay $\tau = \tau_0$. Figure 6 confirms that this peak recovery is robust as τ_0 is varied over $\{20, 60, 120\}$ d for the EXP series: both `nufftcf` and `pyZDCF` recover peaks consistent with the true delay (marked by the vertical dashed line), with `nufftcf`'s continuous curve making the peak location and shape considerably easier to read off than `pyZDCF`'s handful of bins per panel.

Figure 7 presents the physically coupled OU-process test of Equation 8, for which the analytical CCF (Equation 9) is overlaid as a dashed black curve. Both `nufftcf` kernels closely follow the analytical exponential peak at all four coupling strengths $\rho = 0.1, 0.4, 0.7, 0.95$, correctly recovering both the peak location $\tau_0 = 60$ d and its amplitude ρ , including in the weakly coupled, most noise-dominated case ($\rho = 0.1$). The `pyZDCF` points are statistically consistent with the same curve but, again, considerably sparser, particularly away from the peak where the lower pair density per equal-population bin widens the `pyZDCF` error bars.

Across both CCF test cases, the key practical difference between the two estimators is the same one seen

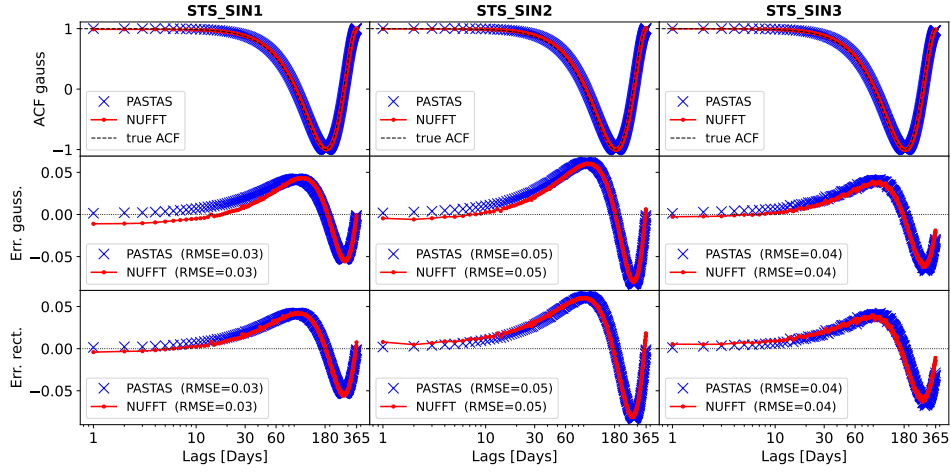


FIG. 2.— ACF of the `STS_SIN1-3` series: signal (top row), `pastas` vs. `nuftcf` Gaussian-kernel ACF against the analytical truth (second row), and residuals for the Gaussian and rectangular kernels (third and fourth rows), with matching RMSE values for both estimators in every panel.

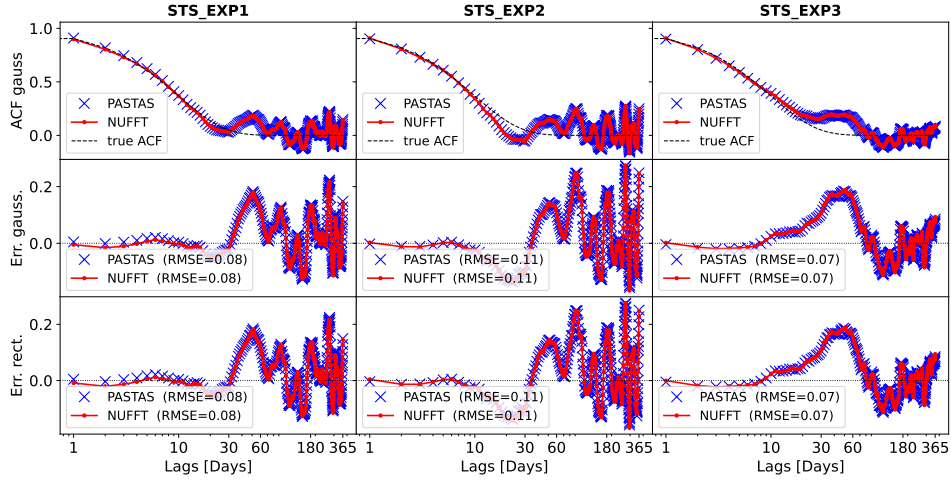


FIG. 3.— Same as Figure 3, for the `STS_EXP1-3` series.

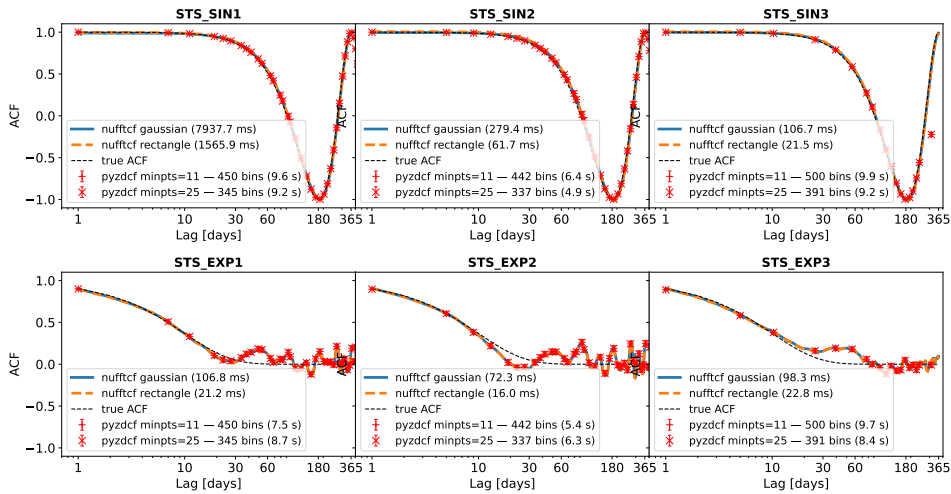


FIG. 4.— Top: ACF of the `STS_SIN1-3` series: signal (top row) and ACF (bottom row), comparing `nufftcf` (Gaussian and rectangular kernels, with per-series computation time) to `pyzdcf` (`minpts=11` and `25`, with number of returned bins and computation time) and to the analytical truth. Bottom: Same for the `STS_EXP1-3` series.

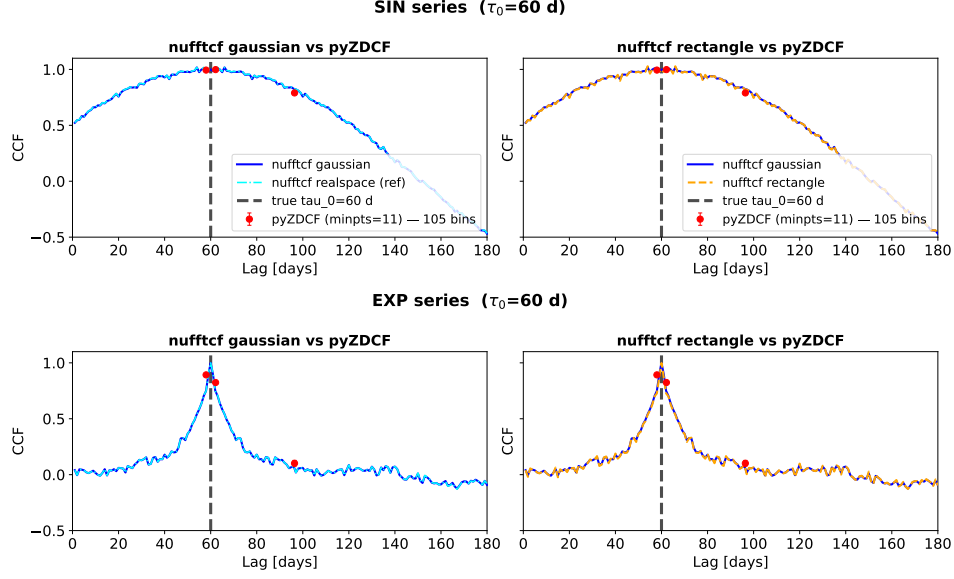


FIG. 5.— Top: CCF of the shifted-timestamp STS_SIN test ($\tau_0 = 60$ d): **nufftcf** Gaussian vs. its own real-space reference (left) and vs. its rectangular kernel (right), both compared to **pyZDCF**. Bottom: Same legend for the shifted-timestamp STS_EXP test.

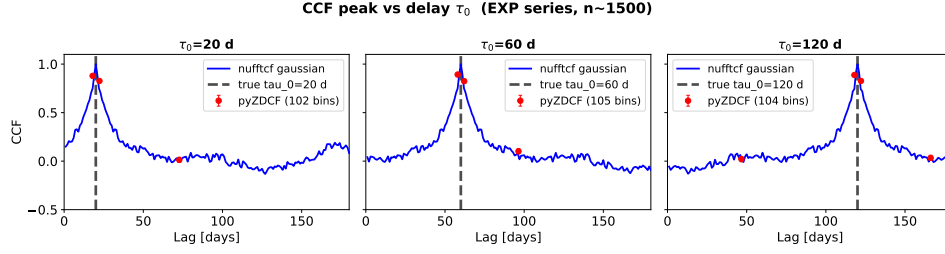


FIG. 6.— CCF peak recovery for the shifted-timestamp EXP network test as the injected delay τ_0 varies over $\{20, 60, 120\}$ d ($n \sim 1500$ points per network): **nufftcf** Gaussian vs. **pyZDCF**, with the true delay marked by the vertical dashed line.

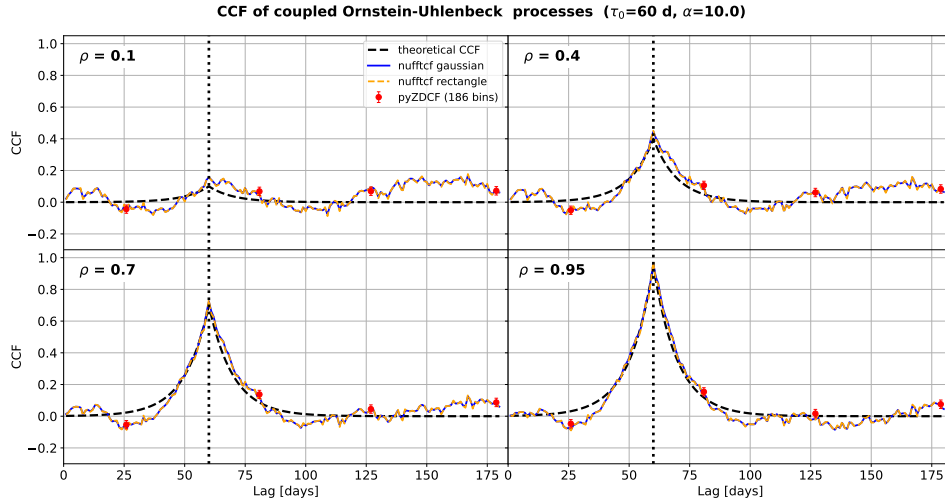


FIG. 7.— CCF of the coupled Ornstein-Uhlenbeck processes of Equation 8 ($\tau_0 = 60$ d, $\alpha = 10$ d), for coupling strengths $\rho = 0.1, 0.4, 0.7, 0.95$: **nufftcf** (Gaussian and rectangular kernels) and **pyZDCF** against the analytical CCF of Equation 9.

for the ACF: `nufftcf` delivers the cross-correlation at *every* requested lag on a uniform grid, whereas `pyZDCF` delivers a much coarser, data-dependent set of lags. This is not an implementation detail but a direct consequence of `pyZDCF`'s equal-population binning strategy (Section 2.2): by design, each `pyZDCF` bin is sized to contain a comparable number of observation pairs, so that its statistical precision is roughly uniform across lags, at the cost of a lag resolution that varies with local pair density and is, in all cases studied here, far coarser than the per-lag, kernel-weighted estimate `nufftcf` evaluates directly at each point of a user-chosen lag grid.

4. TIMING BENCHMARKS

TODO: draft this section from `benchmark/` results (`nufftcf` NUFFT vs `nufftcf` real-space vs `pastas`, ACF timing vs n ; include scaling plot).

[**TODO:** timing benchmark figure(s) and discussion of scaling with n .]

5. DISCUSSION

TODO: discuss (i) when to prefer NUFFT vs real-space

vs classic-FFT, (ii) the spectral-leakage bias and how the number of Fourier modes controls it, (iii) guidance for choosing kernel bandwidth h .

6. SUMMARY AND CONCLUSION

TODO.

7. REPRODUCIBLE RESEARCH

The `nufftcf` package is available at <https://github.com/jecampagne/nufftcf> under the MIT license, together with the notebooks and benchmark scripts used to produce the comparisons and timings in this paper. **TODO:** add PyPI We have used `pastas` 1.14.0, while concerning `pyZDCF` we have made a fork of the <https://github.com/LSST-sersag/pyzdcf> repository to allow the notebooks to be run on the Google Colab platform (Google 2026).

LARGE LANGUAGE MODEL USE DISCLOSURE

[**TODO.**]

ACKNOWLEDGMENTS

We thank the developers of `FINUFFT`, `pastas`, and `pyZDCF`, against which `nufftcf` is validated.

REFERENCES

- Alexander, T. 1997, in *Astrophysics and Space Science Library*, Vol. 218, *Astronomical Time Series*, ed. D. Maoz, A. Sternberg, & E. M. Leibowitz, 163, doi: [10.1007/978-94-015-8941-3_14](https://doi.org/10.1007/978-94-015-8941-3_14)
- Alexander, T. 1997, in *Astronomical Time Series*, ed. D. Maoz, A. Sternberg, & E. M. Leibowitz, Vol. 218 (Springer)
- Barnett, A. H., Magland, J. F., & af Klinteberg, L. 2019, *SIAM Journal on Scientific Computing*, 41, C479, doi: [10.1137/18M120885X](https://doi.org/10.1137/18M120885X)
- Björnstad, O. N., & Falck, W. 2001, *Environmental and Ecological Statistics*, 8, 53, doi: [10.1023/A:1009601932481](https://doi.org/10.1023/A:1009601932481)
- Collenteur, R. A., Bakker, M., Caljé, R., Klop, S. A., & Schaars, F. 2019, *Groundwater*, 57, 877, doi: [10.1111/gwat.12925](https://doi.org/10.1111/gwat.12925)
- Edelson, R. A., & Krolik, J. H. 1988, *ApJ*, 333, 646, doi: [10.1086/166773](https://doi.org/10.1086/166773)
- Edelson, R. A., & Krolik, J. H. 1988, *The Astrophysical Journal*, 333, 646, doi: [10.1086/166773](https://doi.org/10.1086/166773)
- Fisher, R. A. 1915, *Biometrika*, 10, 507, doi: [10.2307/2331838](https://doi.org/10.2307/2331838)
- Google. 2026, Google Colaboratory, <https://colab.research.google.com/>
- Jankov, I., Kovačević, A. B., Ilić, D., et al. 2022a, *Astronomische Nachrichten*, 343, e210090, doi: [10.1002/asna.20210090](https://doi.org/10.1002/asna.20210090)
- Jankov, I., Kovačević, A. B., Ilić, D., Sánchez-Sáez, P., & Nikutta, R. 2022b, `pyZDCF`: Initial Release, doi: [10.5281/zenodo.7253034](https://doi.org/10.5281/zenodo.7253034)
- Kovačević, A., Popović, L. Č., Shapovalova, A. I., et al. 2014, *Advances in Space Research*, 54, 1414, doi: [10.1016/j.asr.2014.06.025](https://doi.org/10.1016/j.asr.2014.06.025)
- Kreutzer, L. T., Gillen, E., Briegal, J. T., & Queloz, D. 2023, *MNRAS*, 522, 5049, doi: [10.1093/mnras/stad1223](https://doi.org/10.1093/mnras/stad1223)
- Lam, S. K., Pitrou, A., & Seibert, S. 2015, in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, LLVM '15* (New York, NY, USA: Association for Computing Machinery), doi: [10.1145/2833157.2833162](https://doi.org/10.1145/2833157.2833162)
- Lomb, N. R. 1976, *Ap&SS*, 39, 447, doi: [10.1007/BF00648343](https://doi.org/10.1007/BF00648343)
- Rehfeld, K., Marwan, N., Heitzig, J., & Kurths, J. 2011, *Nonlinear Processes in Geophysics*, 18, 389, doi: [10.5194/npg-18-389-2011](https://doi.org/10.5194/npg-18-389-2011)
- Sánchez-Sáez, P., Lira, H., Martí, L., et al. 2021, *AJ*, 162, 206, doi: [10.3847/1538-3881/ac1426](https://doi.org/10.3847/1538-3881/ac1426)
- Scargle, J. D. 1982, *ApJ*, 263, 835, doi: [10.1086/160554](https://doi.org/10.1086/160554)
- . 1989, *ApJ*, 343, 874, doi: [10.1086/167757](https://doi.org/10.1086/167757)
- Shapovalova, A. I., Popović, L. Č., Afanasiev, V. L., et al. 2019, *MNRAS*, 485, 4790, doi: [10.1093/mnras/stz692](https://doi.org/10.1093/mnras/stz692)
- Stoffer, D. 2026, *astsa: Applied Statistical Time Series Analysis*. <https://CRAN.R-project.org/package=astsa>
- Stoica, P., Li, J., & He, H. 2009, *Trans. Sig. Proc.*, 57, 843–858, doi: [10.1109/TSP.2008.2008973](https://doi.org/10.1109/TSP.2008.2008973)
- VanderPlas, J. T. 2018, *ApJS*, 236, 16, doi: [10.3847/1538-4365/aab766](https://doi.org/10.3847/1538-4365/aab766)
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, *Nature Methods*, 17, 261, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)
- Welsh, W. 2024, in *AAS/High Energy Astrophysics Division*, Vol. 21, *AAS High Energy Astrophysics Division Meeting #21*, 102.18

APPENDIX

A. NUFFT-BASED ACF/CCF ALGORITHMS

This appendix gives the complete pseudocode underlying the NUFFT-based estimators sketched in Section 2.6 (`nufft_ccf.py` and `nufft_acf.py`), evaluating the Wiener–Khinchin identity of Equation 2 through a type-1 NUFFT (nonuniform samples $\rightarrow$ uniform frequency grid) followed by a type-2 NUFFT (uniform frequency grid $\rightarrow$ requested lags). Both weighting kernels supported by `nufftcf` (Gaussian, rectangular; see Equations 4–5) share the same algorithmic structure and differ only in the smoothing operator $\mathcal{S}_h$ (Gaussian filter

of width h , or box filter of size $\text{round}(2h)$) applied to the raw NUFFT output, and in the matching pair-count function $b(\cdot)$ from Appendix A.2. We therefore present each estimator as a single, kernel-parameterized algorithm rather than duplicating it for each kernel.

A.1. Cross and Autocorrelation functions

Algorithm 1 gives the general NUFFT-based CCF estimator: it evaluates the Wiener–Khinchin identity through a type-1 NUFFT on each series, forms the cross-spectrum $\hat{X}^*(f)\hat{Y}(f)$, evaluates it at the requested lags by a type-2 NUFFT, smooths the result with the chosen kernel $\mathcal{S}_h$, and normalizes it to a Pearson correlation in $[-1, 1]$ by the matching effective pair count (Algorithm 4)

and by the geometric mean of the two series' ACF values at lag 0, $\sqrt{\text{ACF}_x(0) \cdot \text{ACF}_y(0)}$, each obtained from Algorithm 2.

The ACF (Algorithm 3) is the special case $s = t$, $y = x$ of this same estimator, and reduces to a slightly simpler procedure on three points. First, since a single series is involved, the power spectrum $|\hat{X}(f)|^2$ is real and even (Hermitian), so only one type-1 NUFFT call is needed and the $\pm i$ sign convention that must be fixed carefully for the CCF cross-spectrum plays no role here. Second, the effective pair count reduces to Algorithm 4 evaluated with $s = t$. Third, the normalization is simpler: rather than the geometric mean of two scales $\text{ACF}_x(0) \cdot \text{ACF}_y(0)$ computed separately (Algorithm 2), the ACF estimate is normalized directly by the smoothed value at lag 0, obtained as the first entry of the same evaluation array — the interior-position trick of Algorithm 2 is unnecessary here because the ACF is even about $\tau = 0$.

A.2. Effective pair count algorithms

The pair-count functions $b(\cdot)$ (module `kernels.py`) compute the *effective* number of pairs contributing to each lag — the normalization denominator of Equation 3. They are implemented in `numba` using a two-pointer scan: because t (and s) are sorted ascending, the window $[\text{lo}, \text{hi}]$ associated with the pair-matching center advances monotonically as the outer index increases, reducing the cost from $O(n^2)$ to $O(n)$ per lag, and to $O(n \log n)$ overall once parallelized (`prange`) across lags.

We give the *cross* (CCF) version below, which reduces to the ACF case by setting $s = t$ — exactly the relationship between `compute_b_gaussian_cross/compute_b_rectangle_cross` and `compute_b_gaussian/compute_b_rectangle` in the code.

ACF case: setting $s = t$, $n_y = n_x$ recovers exactly `compute_b_gaussian` and `compute_b_rectangle`, with center $= t_j^x + \tau_k$ (pair condition $t_i^x - t_j^x \approx \tau_k$) in place of center $= t_j^y - \tau_k$ (pair condition $t_j^y - t_i^x \approx \tau_k$) — only the window-center definition differs between the two cases.

A.3. Implementation choices dictated by FINUFFT

Algorithms 1–3 rely on FINUFFT for both nonuniform transforms (`nufft1d1`, type 1, and `nufft1d2`, type 2). Several implementation choices follow directly from this library's constraints and conventions; we detail them here, complementing the $O(n)$ -per-lag `numba` kernels of Appendix A.2.

Time domain $[0, 2\pi)$.— FINUFFT requires nonuniform points to lie in the standard domain $[-\pi, \pi)$ or $[0, 2\pi)$, so the user-supplied sample times t (in days, or any other unit) must first be rescaled. For the ACF, a single series is involved and its own extent is used: $\tilde{t} = \frac{t - \min t}{\max t - \min t} \cdot 2\pi$. For the CCF, however, *two* series t and s , potentially shifted and with different supports, must be placed on the *same* frequency grid for the cross-product $\hat{F}_1 \odot \hat{F}_2$ to be meaningful: Algorithm 1 therefore uses the *union* of the two time spans, $t_{\min} = \min(\min t, \min s)$, $\text{span} = \max(\max t, \max s) - t_{\min}$, and applies the same affine map to t , s , and to the requested lags τ . Because lags have units of time, they are rescaled identically, $\tilde{\tau} = \tau / \text{span} \cdot 2\pi$, which is what allows the type-2

output to be read off directly at the desired lags without any further inverse transform.

Sign convention and the non-Hermitian cross-spectrum.— With `isign=+1`, FINUFFT computes $f_j = \sum_k F_k e^{-ikx_j}$ (negative exponent). For the ACF, the power spectrum $|\hat{X}(f)|^2$ is real and even (Hermitian), so the sign convention has no effect: $M = \hat{F} \odot \hat{F}$. For the CCF, however, the cross-spectrum $\hat{X}^*(f) \hat{Y}(f)$ is *not* Hermitian in general: the choice $M = \hat{F}_1 \odot \hat{F}_2$ places the correlation peak at the correct lag $+\tau$ (the opposite choice $\hat{F}_1 \odot \hat{F}_2$ would place it at $-\tau$, a mirrored result). This is a pure convention issue, unrelated to numerical precision, but it must be fixed correctly for the sign of the estimated delay to be physical.

Frequency-grid size N_1 .— The parameter N_1 sets the resolution of the type-1 transform (time $\rightarrow$ frequency), and therefore the fidelity of the reconstructed signal ahead of the type-2 evaluation. The default $N_1 = 32 \max(n_x, n_y)$ (a $32\times$ oversampling factor) was validated empirically against the exact real-space estimator (`compute_ccf_gaussian_realspace`; see `kernels.py`): a factor of 32 is enough to bring the two estimators into close agreement for both the Gaussian and rectangular kernels. Higher oversampling yields only marginal further improvement (mainly for strongly periodic signals with the Gaussian kernel) at the cost of increased $O(N_1 \log N_1)$ time and memory. Scaling $N_1 \propto n$, rather than fixing a constant grid size, keeps the frequency grid adequate regardless of series length.

Requested tolerance ε .— The `eps` parameter (default 10^{-9}) sets the accuracy requested from FINUFFT's internal iterative scheme (gridding + FFT + deconvolution; Section 2.5). A looser tolerance (typically 10^{-6} – 10^{-9} in common FINUFFT usage) speeds up the computation but introduces NUFFT-specific numerical noise, distinct from the spectral-leakage bias discussed in Section 5 (a windowing artifact due to irregular or gappy sampling, of order 1–3% on ACF amplitude, independent of ε).

Lag-0 normalization and interior placement.— The smoothing step (Gaussian or box filter) applied to the raw NUFFT output introduces an edge artifact: a point at the *end* of the evaluated array is not smoothed symmetrically, unlike an *interior* point. The CCF peak of interest (at lag τ_0 , interior to the requested `lags` range) *is* an interior point. Dividing this peak by an $\text{ACF}(0)$ scale computed by placing lag 0 at the *first* position of an array (hence smoothed asymmetrically) introduced a systematic 2–3% deficit in the estimated CCF peak. Algorithm 2 corrects this by evaluating $\text{ACF}_x(0)$ on a small, *symmetric* lag array centered on 0 (indices $-n_h, \dots, 0, \dots, n_h$, with $n_h = \max(\lceil 4h \rceil + 2, 5)$, i.e. at least $4h$ of margin on each side), so that the smoothing filter acts there exactly as it does at the peak — a concrete example of a numerical-pipeline artifact (not a property of the underlying signal) that must be neutralized by how the evaluation array is constructed, rather than corrected *a posteriori*.

Prior standardization.— Both x and y are standardized (zero mean, unit variance) before any call to

Algorithm 1: CCF via NUFFT + smoothing (Gaussian or rectangular)

Require : times $t = (t_i^x)_{i=1}^{n_x}$, values $x = (x_i)$ (series 1, t ascending); times $s = (t_j^y)_{j=1}^{n_y}$, values $y = (y_j)$ (series 2, s ascending); requested lags $\{\tau_1, \dots, \tau_K\}$; kernel filter S_h with bandwidth h (σ for the Gaussian kernel, half-width for the rectangular kernel); NUFFT tolerance ε ; frequency-grid size N_1 (default $32 \max(n_x, n_y)$)

Ensure : $c = (c_1, \dots, c_K)$ (CCF estimate, normalized to $[-1, 1]$), $b = (b_1, \dots, b_K)$ (effective pair count)

- 1 $x_{\text{std}} \leftarrow (x - \bar{x})/\text{std}(x)$, $y_{\text{std}} \leftarrow (y - \bar{y})/\text{std}(y)$; ▷ standardize both series
- 2 $t_{\min} \leftarrow \min(\min t, \min s)$, $\text{span} \leftarrow \max(\max t, \max s) - t_{\min}$; ▷ common (union) time span
- 3 $\tilde{t} \leftarrow \frac{t - t_{\min}}{\text{span}} 2\pi$, $\tilde{s} \leftarrow \frac{s - t_{\min}}{\text{span}} 2\pi$, $\tilde{\tau} \leftarrow \frac{(0, \tau_1, \dots, \tau_K)}{\text{span}} 2\pi$; ▷ lag 0 prepended for the normalization step below
- 4 $\hat{F}_1 \leftarrow \text{NUFFT1D1}(\tilde{t}, x_{\text{std}}, N_1, \varepsilon)$; ▷ type 1: time $\rightarrow$ frequency
- 5 $\hat{F}_2 \leftarrow \text{NUFFT1D1}(\tilde{s}, y_{\text{std}}, N_1, \varepsilon)$;
- 6 $M \leftarrow \hat{F}_1 \odot \hat{F}_2$; ▷ cross-spectrum $\hat{X}^*(f)\hat{Y}(f)$, not Hermitian in general
- 7 $c_{\text{raw}} \leftarrow \Re[\text{NUFFT1D2}(\tilde{\tau}, M, \varepsilon)]$; ▷ type 2: frequency $\rightarrow$ lags
- 8 $c_{\text{sm}} \leftarrow \mathcal{S}_h(c_{\text{raw}})$; ▷ Gaussian or rectangular filtering
- 9 $b_{\text{cross}} \leftarrow \text{PAIRCOUNT}(t, s, (0, \tau_1, \dots, \tau_K), h)$; ▷ Alg. 4, matching kernel
- 10 $c_{\text{norm}} \leftarrow c_{\text{sm}} \oslash b_{\text{cross}}$; ▷ elementwise division
- 11 $\text{scale}_x \leftarrow \text{ACFSCALEATLAG0}(t, x_{\text{std}}, t_{\min}, \text{span}, N_1, \varepsilon, h)$; ▷ Alg. 2
- 12 $\text{scale}_y \leftarrow \text{ACFSCALEATLAG0}(s, y_{\text{std}}, t_{\min}, \text{span}, N_1, \varepsilon, h)$;
- 13 $\text{scale} \leftarrow \sqrt{\text{scale}_x \cdot \text{scale}_y}$;
- 14 $c \leftarrow c_{\text{norm}}[2:] / \text{scale}$; ▷ drop the lag-0 entry (indexing starts at 1)
- 15 $b \leftarrow b_{\text{cross}}[2:]$;
- 16 **return** c, b

Algorithm 2: ACF scale at lag 0 (Pearson normalization) — ACFSCALEATLAG0

Require : $t, x_{\text{std}}, t_{\min}, \text{span}$ (shared by both series), N_1, ε , bandwidth h

Ensure : scalar estimate of $\text{ACF}_x(0)$

- 1 $\tilde{t} \leftarrow (t - t_{\min})/\text{span} \cdot 2\pi$;
- 2 $\hat{F} \leftarrow \text{NUFFT1D1}(\tilde{t}, x_{\text{std}}, N_1, \varepsilon)$, $P \leftarrow |\hat{F}|^2$;
- 3 $n_h \leftarrow \max(\lceil 4h \rceil + 2, 5)$; $\tau_{\text{sym}} \leftarrow (-n_h, \dots, -1, 0, 1, \dots, n_h)$; ▷ lag 0 placed at an *interior* array position
- 4 $\tilde{\tau}_{\text{sym}} \leftarrow \tau_{\text{sym}}/\text{span} \cdot 2\pi$;
- 5 $c_{\text{raw}} \leftarrow \Re[\text{NUFFT1D2}(\tilde{\tau}_{\text{sym}}, P, \varepsilon)]$;
- 6 $c_{\text{sm}} \leftarrow \mathcal{S}_h(c_{\text{raw}})$; ▷ same smoothing as Alg. 1, applied symmetrically
- 7 $b_{\text{sym}} \leftarrow \text{PAIRCOUNT}(t, t, \tau_{\text{sym}}, h)$; ▷ Alg. 4 with $s = t$
- 8 **return** $c_{\text{sm}}[n_h + 1] / \max(b_{\text{sym}}[n_h + 1], 10^{-16})$

Algorithm 3: ACF via NUFFT + smoothing (Gaussian or rectangular)

Require : times t , values x ; lags $\{\tau_1, \dots, \tau_K\}$; bandwidth h ; ε ; N_1 (default $32n$)

Ensure : $c = (c_1, \dots, c_K)$, $b = (b_1, \dots, b_K)$

- 1 $x_{\text{std}} \leftarrow (x - \bar{x})/\text{std}(x)$;
- 2 $\tilde{t} \leftarrow \frac{t - \min t}{\max t - \min t} 2\pi$, $\tilde{\tau} \leftarrow \frac{(0, \tau_1, \dots, \tau_K)}{\max t - \min t} 2\pi$;
- 3 $\hat{F} \leftarrow \text{NUFFT1D1}(\tilde{t}, x_{\text{std}}, N_1, \varepsilon)$;
- 4 $M \leftarrow \hat{F} \odot \hat{F} = |\hat{F}|^2$; ▷ real and even: the Hermitian case, no sign ambiguity
- 5 $c_{\text{raw}} \leftarrow \Re[\text{NUFFT1D2}(\tilde{\tau}, M, \varepsilon)]$;
- 6 $c_{\text{sm}} \leftarrow \mathcal{S}_h(c_{\text{raw}})$;
- 7 $b_{\text{eval}} \leftarrow \text{PAIRCOUNT}(t, t, (0, \tau_1, \dots, \tau_K), h)$; ▷ Alg. 4, $s = t$
- 8 $c_{\text{eval}} \leftarrow c_{\text{sm}} \oslash b_{\text{eval}}$;
- 9 $c \leftarrow c_{\text{eval}}[2:] / c_{\text{eval}}[1]$; ▷ normalize by the lag-0 value
- 10 $b \leftarrow b_{\text{eval}}[2:]$;
- 11 **return** c, b

FINUFFT. This is not dictated by FINUFFT itself, but ensures that the NUFFT's internal scale (which depends on input amplitude) is consistent between the two compared series; combined with the final division by $\sqrt{\text{ACF}_x(0) \cdot \text{ACF}_y(0)}$, it guarantees $c \in [-1, 1]$ under the Pearson convention, regardless of the original physical scale of either series.

This paper was built using the Open Journal of As-

trophysics L^AT_EX template. The OJA is a journal which provides fast and easy peer review for new papers in the **astro-ph** section of the arXiv, making the reviewing process simpler for authors and referees alike. Learn more at <http://astro.theoj.org>.

Algorithm 4: PAIRCOUNT — Gaussian (weighted) kernel, general cross case

Require : $t = (t_i^x)_{i=1}^{n_x}$ sorted, $s = (t_j^y)_{j=1}^{n_y}$ sorted, lags $\{\tau_k\}_{k=1}^K$, bandwidth h
Ensure : $b = (b_k)_{k=1}^K$

```

1 for  $k \leftarrow 1$  to  $K$  in parallel
2    $\text{lo} \leftarrow 0, \text{hi} \leftarrow 0, b_k \leftarrow 0;$ 
3   for  $j \leftarrow 1$  to  $n_y$  do
4      $\triangleright$  increasing  $j \Rightarrow$  increasing center  $\Rightarrow$  lo/hi only advance
4      $\text{center} \leftarrow t_j^y - \tau_k;$ 
5     while  $\text{lo} < n_x$  and  $t_{\text{lo}}^x < \text{center} - 6h$  do
6        $\text{lo} \leftarrow \text{lo} + 1;$ 
7     while  $\text{hi} < n_x$  and  $t_{\text{hi}}^x < \text{center} + 6h$  do
8        $\text{hi} \leftarrow \text{hi} + 1;$ 
9     for  $i \leftarrow \text{lo}$  to  $\text{hi} - 1$  do
10       $b_k \leftarrow b_k + \frac{1}{h\sqrt{2\pi}} \exp\left(-\frac{(t_i^x - \text{center})^2}{2h^2}\right);$ 
11    $b_k \leftarrow \max(b_k, 10^{-16});$   $\triangleright$  avoids division by zero at sparsely-populated lags
12 return  $b$ 

```

Algorithm 5: PAIRCOUNT — rectangular (counting) kernel, general cross case

Require : $t = (t_i^x)_{i=1}^{n_x}$ sorted, $s = (t_j^y)_{j=1}^{n_y}$ sorted, lags $\{\tau_k\}_{k=1}^K$, half-width h
Ensure : $b = (b_k)_{k=1}^K$

```

1 for  $k \leftarrow 1$  to  $K$  in parallel
2    $\text{lo} \leftarrow 0, \text{hi} \leftarrow 0, b_k \leftarrow 0;$ 
3   for  $j \leftarrow 1$  to  $n_y$  do
4      $\text{center} \leftarrow t_j^y - \tau_k;$ 
5     while  $\text{lo} < n_x$  and  $t_{\text{lo}}^x < \text{center} - h$  do
6        $\text{lo} \leftarrow \text{lo} + 1;$ 
7      $\text{hi} \leftarrow \max(\text{hi}, \text{lo});$ 
8     while  $\text{hi} < n_x$  and  $t_{\text{hi}}^x \leq \text{center} + h$  do
9        $\text{hi} \leftarrow \text{hi} + 1;$ 
10     $b_k \leftarrow b_k + (\text{hi} - \text{lo});$   $\triangleright$  direct count, no weighted sum:  $O(1)$  per  $j$ 
11     $b_k \leftarrow \max(b_k, 10^{-16});$ 
12 return  $b$ 

```
